

ARTIFICIAL INTELLIGENCE

CO4 – ASSESSMENT TOOL 1

INTEGRATED EXPERIMENTS

Name	T. Niranjana Kumar
Reg. No.	192473007
Course Code	CSA1711

EXPERIMENT 1 – DECISION TREE CLASSIFICATION

Objective

Implement and evaluate a Decision Tree classifier on the Iris dataset using Python and a machine learning library.

(a) Dataset Loading, Pre-processing and Train-Test Split

Load the dataset, check missing values, display the first 5 rows and data types, and split the data into training and testing sets.

Python Code

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name="target")

print(X.head())
print(X.dtypes)
print(X.isnull().sum())

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

Explanation

The Iris dataset contains numeric features. Missing values are checked before training. The data is divided into 80% training and 20% testing sets.

(b) Build the Decision Tree and Print the Tree Structure

Python Code

```
from sklearn.tree import DecisionTreeClassifier, export_text

model = DecisionTreeClassifier(criterion="gini", random_state=42)
model.fit(X_train, y_train)

print(export_text(model, feature_names=list(X.columns)))

root_index = model.tree_.feature[0]
```

```
print("Root Node:", X.columns[root_index])
print("Threshold:", model.tree_.threshold[0])
```

Root Node Justification

The root node is the first feature selected by the trained Decision Tree. Using the Gini criterion, it is selected because it provides the best impurity reduction at the first split. The exact feature is displayed when the code is executed.

(c) Evaluate the Model

Python Code

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

misclassified = X_test[y_test.values != y_pred].copy()
misclassified["Actual"] = y_test[y_test.values != y_pred].values
misclassified["Predicted"] = y_pred[y_test.values != y_pred]
print("Misclassified Instances:")
print(misclassified.head(2))
```

Performance Comment

The accuracy score shows overall performance. The confusion matrix shows correct and incorrect predictions. The classification report gives precision, recall and F1-score. Up to two misclassified instances are printed from the test data.

EXPERIMENT 2 – REINFORCEMENT LEARNING: Q-LEARNING

Objective

Implement Q-Learning for a simple 4×4 Grid World environment and observe the agent's learned policy.

(a) Environment, Actions, Rewards and Q-table

States: 16 cells in a 4×4 grid. Actions: Up, Down, Left and Right. Rewards: +10 for Goal, −1 for each step and −5 for an obstacle.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
import random

ROWS, COLS = 4, 4
N_STATES = 16
N_ACTIONS = 4
START, GOAL = 0, 15
OBSTACLES = {5, 10}

alpha = 0.1
gamma = 0.9
epsilon = 0.2

Q = np.zeros((N_STATES, N_ACTIONS))
print(Q)
```

Hyperparameters

$\alpha = 0.1$ (learning rate), $\gamma = 0.9$ (discount factor), $\epsilon = 0.2$ (exploration rate). The initial Q-table contains zeros.

(b) Run Q-Learning for 100 Episodes

Python Code

```
def move(state, action):
    row, col = divmod(state, COLS)
    if action == 0: row = max(0, row-1)
    elif action == 1: row = min(ROWS-1, row+1)
    elif action == 2: col = max(0, col-1)
    else: col = min(COLS-1, col+1)
    ns = row * COLS + col
```

```

    reward = 10 if ns == GOAL else (-5 if ns in OBSTACLES else -1)
    return ns, reward

rewards = []
snapshots = {}
states_to_show = [0, 6]

for episode in range(1, 101):
    state = START
    total = 0
    for _ in range(100):
        action = random.randrange(N_ACTIONS) if random.random() < epsilon else
np.argmax(Q[state])
        ns, reward = move(state, action)
        Q[state, action] += alpha * (reward + gamma*np.max(Q[ns]) - Q[state, action])
        state, total = ns, total + reward
        if state == GOAL: break
    rewards.append(total)
    if episode in [1, 50, 100]:
        snapshots[episode] = Q[states_to_show].copy()

for ep in [1, 50, 100]:
    print("Episode", ep, snapshots[ep])

```

Explanation

Q-values for two representative states are recorded at Episodes 1, 50 and 100. The changing values show how the Q-table evolves through repeated learning.

(c) Final Learned Policy and Cumulative Reward

Python Code

```

symbols = ["↑", "↓", "←", "→"]
policy = []
for state in range(N_STATES):
    if state == GOAL:
        policy.append("G")
    elif state in OBSTACLES:
        policy.append("X")
    else:
        policy.append(symbols[np.argmax(Q[state])])

for r in range(ROWS):
    print(policy[r*COLS:(r+1)*COLS])

```

```
plt.figure(figsize=(8,4))
plt.plot(range(1,101), rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Cumulative Reward per Episode")
plt.grid(True)
plt.show()
```

Convergence Behaviour

Early episodes contain more exploration, so rewards can fluctuate. Over time, useful actions receive better Q-values. Convergence is indicated when the learned policy becomes stable and rewards generally improve or stabilize.

FINAL CONCLUSION

Experiment 1 implements and evaluates a Decision Tree classifier using accuracy, confusion matrix, classification report and misclassified instances. Experiment 2 implements Q-Learning in a 4×4 Grid World, records Q-value evolution, extracts the final policy and plots cumulative reward per episode.